torch.profiler

回顾安装Torchbench

默认配置torch==2.5.1+cu124，torchvision==0.20.1+cu124

需要安装对应版本torchaudio

```
pip install torchaudio==2.5.1
```

再执行安装过程

```
git clone https://github.com/pytorch/benchmark
cd benchmark
python install.py models="BERT_pytorch, timm_nfnet"
pip install -e .
```

学术加速

```
source /etc/network_turbo
unset http_proxy && unset https_proxy
```

整网运行生成Kernel文件和profiler追踪文件

```
TORCHINDUCTOR_UNIQUE_KERNEL_NAMES=1 TORCHINDUCTOR_BENCHMARK_KERNEL=1 python -u
benchmark/userbenchmark/dynamo/dynamobench/torchbench.py --backend inductor --
float16 --performance --only BERT_pytorch --training --export-profiler-trace
```

全局搜索`/root/benchmark/userbenchmark/dynamo/dynamobench/common.py`文件中的export_profiler_trace，对profiler 捕获生成的两处.json文件路径进行修改：

```
        # name = os.path.join(torch._dynamo.config.base_dir, name)
        name = os.path.join("/root/autodl-tmp/BERT",name)
```

同时定位至speedup = median[0] / median[1]在脚本两处位置添加打印查看eager和compile整网运行的时间

```
    speedup = median[0] / median[1]
    print("eager:"+str(median[0]))
    print("compile:"+str(median[1]))
```

换为推理模式

```
TORCHINDUCTOR_UNIQUE_KERNEL_NAMES=1 TORCHINDUCTOR_BENCHMARK_KERNEL=1 python -u
benchmark/userbenchmark/dynamo/dynamobench/torchbench.py --backend inductor --
float16 --performance --only BERT_pytorch --inference --export-profiler-trace --
output-directory="/root/autodl-tmp/lesson-2/"
```

使用-h查看脚本更多选项功能和帮助信息

```
python benchmark/userbenchmark/dynamo/dynamobench/torchbench.py -h
```

单独研究每个编译模块的性能

```
python xx.py -p
```

保存到日志文件中

```
python xx.py -p > xx.log
```

-p执行同样也会捕获生成.json文件保存在`/tmp/compiled_module_profile.json`

若是空环境，需要安装下面python包依赖

```
pip install pandas psutil pyyaml scipy tqdm tabulate
```

可将.json文件下载至本地后

在谷歌chrome浏览器(或相同内核的其他浏览器EDGE)中地址栏中输入`chrome://tracing`（`edge://tracing/`）上传json文件进行可视化查看

通过json的timeline文件去查看算子的并行情况（以存在算子并行的timm_nfnet网络为例，kernel_41、kernel_35）

```
TORCHINDUCTOR_UNIQUE_KERNEL_NAMES=1 TORCHINDUCTOR_BENCHMARK_KERNEL=1 python -u
benchmark/userbenchmark/dynamo/dynamobench/torchbench.py --backend inductor --
float16 --performance --only timm_nfnet --inference --export-profiler-trace --
output-directory="/root/autodl-tmp/lesson-2/timm_nfnet/"
```

(前面common.py路径设置为了`/root/autodl-tmp/lesson-2`，所以timm_nfnet的.json会在此文件夹下)

torch benchmark脚本中torch.profiler接口查看，
在`/root/benchmark/userbenchmark/dynamo/dynamobench/common.py`文件中全局搜索`torch.profiler.profile`

使用torch.profiler单独分析Kernel

复制网络生成Triton Kernel中耗时较长的一个，以`triton_poi_fused_gelu_7`为例

```
import triton
import triton.language as tl
from triton.compiler.compiler import AttrsDescriptor

from torch._inductor.runtime import triton_helpers, triton_heuristics
from torch._inductor.runtime.triton_helpers import libdevice, math as tl_math
from torch._inductor.runtime.hints import AutotuneHint, ReductionHint, TileHint,
instance_descriptor, DeviceProperties

from torch._dynamo.testing import rand_strided
from torch._C import _cuda_getCurrentRawStream as get_raw_stream
import torch
from torch._inductor.runtime.triton_heuristics import grid, split_scan_grid
```

```
@triton_heuristics.pointwise(
    size_hints=[8388608],
    filename=__file__,
    triton_meta={'signature': {0: '*fp16', 1: '*fp16', 2: 'i32'}, 'device':
DeviceProperties(type='cuda', index=0, cc=86, major=8,
regs_per_multiprocessor=65536, max_threads_per_multi_processor=1536,
multi_processor_count=82), 'constants': {}, 'configs':
[AttrsDescriptor(divisible_by_16=(0, 1, 2), equal_to_1=())]},
    inductor_meta={'autotune_hints': set(), 'kernel_name':
'triton_poi_fused_gelu_7', 'mutated_arg_names': ['in_out_ptr0'], 'no_x_dim':
False, 'num_load': 2, 'num_reduction': 0, 'backend_hash':
'61C517A620B536BFDF947A3BF37F0A6082A6B09AF732A916991DD30BE3883ED9',
'are_deterministic_algorithms_enabled': False, 'assert_indirect_indexing': True,
'autotune_local_cache': True, 'autotune_pointwise': True,
'autotune_remote_cache': None, 'force_disable_caches': False,
'dynamic_scale_rblock': True, 'max_autotune': False, 'max_autotune_pointwise':
False, 'min_split_scan_rblock': 256, 'spill_threshold': 16, 'store_cubin': False,
'kernel_num_gb': 0.025171968},
    min_elem_per_thread=0
)
@triton.jit
def triton_poi_fused_gelu_7(in_out_ptr0, in_ptr0, xnumel, XBLOCK : tl.constexpr):
    xnumel = 6291456
    xoffset = tl.program_id(0) * XBLOCK
    xindex = xoffset + tl.arange(0, XBLOCK)[:]
    xmask = tl.full([XBLOCK], True, tl.int1)
    x2 = xindex
    x0 = xindex % 3072
    tmp0 = tl.load(in_out_ptr0 + (x2), None).to(tl.float32)
    tmp1 = tl.load(in_ptr0 + (x0), None,
eviction_policy='evict_last').to(tl.float32)
    tmp2 = tmp0 + tmp1
    tmp3 = tmp2.to(tl.float32)
    tmp4 = 0.5
    tmp5 = tmp3 * tmp4
    tmp6 = 0.7071067811865476
    tmp7 = tmp3 * tmp6
    tmp8 = libdevice.erf(tmp7)
    tmp9 = 1.0
    tmp10 = tmp8 + tmp9
    tmp11 = tmp5 * tmp10
    tmp12 = tmp11.to(tl.float32)
    tl.store(in_out_ptr0 + (x2), tmp12, None)


def get_args():
    arg_0 = rand_strided((16, 128, 3072), (393216, 3072, 1), device='cuda:0',
dtype=torch.float16)
    arg_1 = rand_strided((3072,), (1,), device='cuda:0', dtype=torch.float16)
    return arg_0, arg_1,


def call(args):
    with torch.cuda._DeviceGuard(0):
        torch.cuda.set_device(0)
```

```
        stream0 = get_raw_stream(0)
        triton_poi_fused_gelu_7.run(*args, 6291456, grid=grid(6291456),
stream=stream0)


def benchmark_all_configs(args):
    with torch.cuda._DeviceGuard(0):
        torch.cuda.set_device(0)
        return triton_poi_fused_gelu_7.benchmark_all_configs(*args, 6291456,
grid=grid(6291456))


if __name__ == '__main__':
    from torch._inductor.runtime.benchmarking import benchmarker

    args = get_args()
    ms = benchmarker.benchmark_gpu(lambda: call(args), rep=40, fast_flush=True)
    num_gb = 0.025171968
    gb_per_s = num_gb / (ms / 1e3)
    print(f"{ms:.3f}ms    {num_gb:.3f}GB    {gb_per_s:.2f}GB/s")
```

进入`/root/miniconda3/lib/python3.12/site-packages/torch/_inductor/runtime/benchmarking.py`

根据调用链查看原有计时方式

benchmark_gpu -> triton_do_bench -> do_bench -> `miniconda3/lib/python3.1/site-packages/triton/testing.py` 使用`start_event.record()`和`end_event.record()`时间戳进行计时

修改时间测量方式

```
if __name__ == '__main__':
    args = get_args()
    torch.cuda.set_device(0)
    print("Warning up ...")
    for _ in range(10):
        call(args)
    torch.cuda.synchronize()

    with torch.profiler.profile(
        activities = [torch.profiler.ProfilerActivity.CPU,
                      torch.profiler.ProfilerActivity.CUDA],
        schedule = torch.profiler.schedule(
            wait=0,warmup=0, active=20, repeat=40
        ),
        record_shapes= True,
        profile_memory=True
    ) as p:
        for _ in range(20):
            call(args)
        torch.cuda.synchronize()
    print(p.key_averages(group_by_input_shape=True).table())
    p.export_chrome_trace("/root/autodl-tmp/BERT/kernel_7.json")
```

通过注释关键语句粗略定位瓶颈，观察时间变化

```
...
# tmp8 = libdevice.erf(tmp7)
tmp8 = tmp7
...
```

HTA可视化工具

VScode安装jupyter插件

从命令面板中**选择解释器命令 (** Ctrl+Shift+P)，搜索jupyter选择“create: New Jupyter Notebook ”

右上角“选择内核”，这里需要使用conda环境，因此需要创建conda环境，并安装HTA工具

```
# 在conda环境中安装HTA工具
pip install HolisticTraceAnalysis
```

然后使用以下代码导入HTA包和trace文件

```
from hta.trace_analysis import TraceAnalysis
analyzer = TraceAnalysis(trace_dir = "/root/autodl-tmp/BERT_pytorch/hta_trace")

from hta.trace_analysis import TraceAnalysis
from glob import glob
import json

for trace in glob("/root/autodl-
tmp/BERT_pytorch/hta_trace/inductor_BERT_pytorch_rank_0.json"):
  with open(trace, 'r') as f:
    trace_obj = json.load(f)
    trace_obj['rank'] = 0

  with open(trace, 'w') as f:
    json.dump(trace_obj, f)

analyzer = TraceAnalysis(trace_dir="/root/autodl-tmp/BERT_pytorch/hta_trace")
```

时间分类

```
time_spent_df = analyzer.get_temporal_breakdown(visualize = True)
```

空闲时间分解

```
idle_time_df = analyzer.get_idle_time_breakdown()
```

Kernel分类

```
kernel_type_metrics_df, kernel_metrics_df = analyzer.get_gpu_kernel_breakdown()
```

内存带宽和队列长度计数器

```
analyzer.generate_trace_with_counters()
```

CUDA Kernel启动分析

```
kernel_info_df = analyzer.get_cuda_kernel_launch_stats()
```

更多帮助信息和使用方法介绍

https://github.com/facebookresearch/HolisticTraceAnalysis

https://pytorch.ac.cn/tutorials/beginner/hta_intro_tutorial.html

NVIDIA性能分析工具

Nsight System & Nsight Compute本地GUI版本安装，选择系统对应版本：

https://developer.nvidia.com/tools-downloads

远程终端命令行CLI版本安装：

系统已经安装CUDA Tookit，通常会自带Nsight Compute，可以不用安装

Nsight System需要进行安装：

```
# 在官网下载
wget https://developer.nvidia.com/downloads/assets/tools/secure/nsight-systems/2025_1/NsightSystems-linux-cli-public-2025.1.1.131-3554042.deb
# 安装deb包
sudo dpkg -i NsightSystems-linux-cli-public-2025.1.1.131-3554042.deb
```

安装完成后，输入`nsys`出现帮助信息，则表明安装成功

```
nsys profile --stats=true -o inference python inference.py
```

主机端下载后进行安装

https://developer.nvidia.com/downloads/assets/tools/secure/nsight-systems/2025_1/nsightsystems-2025.1.1.131-3554042.msi

安装完成后导入从服务器下载的.nsys-rep文件，有不同选项视图可以去查看

使用命令行工具ncu来分析CUDA程序，运行时需要使用管理员权限运行，否则会收到没有权限访问GPU计数器的报错信息。

ncu --set full --target-processes all -o kernel_7 python triton_poi_fused_gelu_7.py

proton性能分析工具

```
import torch

import triton
import triton.language as tl
import triton.profiler as proton
from typing import NamedTuple
```

```
import argparse


def unpack_grid(grid):
    if len(grid) == 1:
        return grid[0], 1, 1
    if len(grid) == 2:
        return grid[0], grid[1], 1
    if len(grid) == 3:
        return grid[0], grid[1], grid[2]


def metadata_fn(
    grid: tuple,
    metadata: NamedTuple,
    args: dict,
):
    grid_x, grid_y, grid_z = unpack_grid(grid)
    num_warps = metadata.num_warps
    num_stages = metadata.num_stages
    cluster_x, cluster_y, cluster_z = metadata.cluster_dims
    shared_memory = metadata.shared
    M, K = args["a_ptr"].shape
    K, N = args["b_ptr"].shape
    return {
        "name":
        f"matmul_<grid:{grid_x}x{grid_y}x{grid_z}>_<cluster:
{cluster_x}x{cluster_y}x{cluster_z}>_<warps:{num_warps}>_<shared:
{shared_memory}>_<stages:{num_stages}>",
        "flops": 2 * M * N * K,
        "bytes": (M * N + N * K + K * M) * args["a_ptr"].element_size(),
    }


@triton.autotune(
    configs=[
        triton.Config(
            {
                "BLOCK_SIZE_M": 128,
                "BLOCK_SIZE_N": 256,
                "BLOCK_SIZE_K": 64,
                "GROUP_SIZE_M": 8,
            },
            num_stages=3,
            num_warps=8,
        ),
        triton.Config(
            {
                "BLOCK_SIZE_M": 64,
                "BLOCK_SIZE_N": 256,
                "BLOCK_SIZE_K": 32,
                "GROUP_SIZE_M": 8,
            },
            num_stages=4,
            num_warps=4,
        ),
```

```
        triton.Config(
            {
                "BLOCK_SIZE_M": 128,
                "BLOCK_SIZE_N": 128,
                "BLOCK_SIZE_K": 32,
                "GROUP_SIZE_M": 8,
            },
            num_stages=4,
            num_warps=4,
        ),
        triton.Config(
            {
                "BLOCK_SIZE_M": 128,
                "BLOCK_SIZE_N": 64,
                "BLOCK_SIZE_K": 32,
                "GROUP_SIZE_M": 8,
            },
            num_stages=4,
            num_warps=4,
        ),
        triton.Config(
            {
                "BLOCK_SIZE_M": 64,
                "BLOCK_SIZE_N": 128,
                "BLOCK_SIZE_K": 32,
                "GROUP_SIZE_M": 8,
            },
            num_stages=4,
            num_warps=4,
        ),
        triton.Config(
            {
                "BLOCK_SIZE_M": 128,
                "BLOCK_SIZE_N": 32,
                "BLOCK_SIZE_K": 32,
                "GROUP_SIZE_M": 8,
            },
            num_stages=4,
            num_warps=4,
        ),
        triton.Config(
            {
                "BLOCK_SIZE_M": 64,
                "BLOCK_SIZE_N": 32,
                "BLOCK_SIZE_K": 32,
                "GROUP_SIZE_M": 8,
            },
            num_stages=5,
            num_warps=2,
        ),
        triton.Config(
            {
                "BLOCK_SIZE_M": 32,
                "BLOCK_SIZE_N": 64,
                "BLOCK_SIZE_K": 32,
                "GROUP_SIZE_M": 8,
```

```
            },
            num_stages=5,
            num_warps=2,
        ),
    ],
    key=["M", "N", "K"],
)
@triton.jit(launch_metadata=metadata_fn)
def matmul_kernel(
        # Pointers to matrices
        a_ptr, b_ptr, c_ptr,
        # Matrix dimensions
        M, N, K,
        # The stride variables represent how much to increase the ptr by when
moving by 1
        # element in a particular dimension. E.g. `stride_am` is how much to
increase `a_ptr`
        # by to get the element one row down (A has M rows).
        stride_am, stride_ak,  #
        stride_bk, stride_bn,  #
        stride_cm, stride_cn,
        # Meta-parameters
        BLOCK_SIZE_M: tl.constexpr, BLOCK_SIZE_N: tl.constexpr, BLOCK_SIZE_K:
tl.constexpr,  #
        GROUP_SIZE_M: tl.constexpr,  #
        ACTIVATION: tl.constexpr,  #
):
    """Kernel for computing the matmul C = A x B.
    A has shape (M, K), B has shape (K, N) and C has shape (M, N)
    """
    # -----------------------------------------------------------
    # Map program ids `pid` to the block of C it should compute.
    # This is done in a grouped ordering to promote L2 data reuse.
    # See above `L2 Cache Optimizations` section for details.
    pid = tl.program_id(axis=0)
    num_pid_m = tl.cdiv(M, BLOCK_SIZE_M)
    num_pid_n = tl.cdiv(N, BLOCK_SIZE_N)
    num_pid_in_group = GROUP_SIZE_M * num_pid_n
    group_id = pid // num_pid_in_group
    first_pid_m = group_id * GROUP_SIZE_M
    group_size_m = min(num_pid_m - first_pid_m, GROUP_SIZE_M)
    pid_m = first_pid_m + (pid % group_size_m)
    pid_n = (pid % num_pid_in_group) // group_size_m

    # ----------------------------------------------------------
    # Create pointers for the first blocks of A and B.
    # We will advance this pointer as we move in the K direction
    # and accumulate
    # `a_ptrs` is a block of [BLOCK_SIZE_M, BLOCK_SIZE_K] pointers
    # `b_ptrs` is a block of [BLOCK_SIZE_K, BLOCK_SIZE_N] pointers
    # See above `Pointer Arithmetic` section for details
    offs_am = (pid_m * BLOCK_SIZE_M + tl.arange(0, BLOCK_SIZE_M)) % M
    offs_bn = (pid_n * BLOCK_SIZE_N + tl.arange(0, BLOCK_SIZE_N)) % N
    offs_k = tl.arange(0, BLOCK_SIZE_K)
    a_ptrs = a_ptr + (offs_am[:, None] * stride_am + offs_k[None, :] * stride_ak)
    b_ptrs = b_ptr + (offs_k[:, None] * stride_bk + offs_bn[None, :] * stride_bn)
```

```
    # -----------------------------------------------------------
    # Iterate to compute a block of the C matrix.
    # We accumulate into a `[BLOCK_SIZE_M, BLOCK_SIZE_N]` block
    # of fp32 values for higher accuracy.
    # `accumulator` will be converted back to fp16 after the loop.
    accumulator = tl.zeros((BLOCK_SIZE_M, BLOCK_SIZE_N), dtype=tl.float32)
    for k in range(0, tl.cdiv(K, BLOCK_SIZE_K)):
        # Load the next block of A and B, generate a mask by checking the K
dimension.
        # If it is out of bounds, set it to 0.
        a = tl.load(a_ptrs, mask=offs_k[None, :] < K - k * BLOCK_SIZE_K,
other=0.0)
        b = tl.load(b_ptrs, mask=offs_k[:, None] < K - k * BLOCK_SIZE_K,
other=0.0)
        # We accumulate along the K dimension.
        accumulator += tl.dot(a, b)
        # Advance the ptrs to the next K block.
        a_ptrs += BLOCK_SIZE_K * stride_ak
        b_ptrs += BLOCK_SIZE_K * stride_bk
    # You can fuse arbitrary activation functions here
    # while the accumulator is still in FP32!
    if ACTIVATION == "leaky_relu":
        accumulator = leaky_relu(accumulator)
    c = accumulator.to(tl.float16)

    # -----------------------------------------------------------
    # Write back the block of the output matrix C with masks.
    offs_cm = pid_m * BLOCK_SIZE_M + tl.arange(0, BLOCK_SIZE_M)
    offs_cn = pid_n * BLOCK_SIZE_N + tl.arange(0, BLOCK_SIZE_N)
    c_ptrs = c_ptr + stride_cm * offs_cm[:, None] + stride_cn * offs_cn[None, :]
    c_mask = (offs_cm[:, None] < M) & (offs_cn[None, :] < N)
    tl.store(c_ptrs, c, mask=c_mask)


# We can fuse `leaky_relu` by providing it as an `ACTIVATION` meta-parameter in
`_matmul`.
@triton.jit
def leaky_relu(x):
    x = x + 1
    return tl.where(x >= 0, x, 0.01 * x)


# %%
# We can now create a convenience wrapper function that only takes two input
tensors,
# and (1) checks any shape constraint; (2) allocates the output; (3) launches the
above kernel.


def matmul(a, b, activation=""):
    # Check constraints.
    assert a.shape[1] == b.shape[0], "Incompatible dimensions"
    assert a.is_contiguous(), "Matrix A must be contiguous"
    assert b.is_contiguous(), "Matrix B must be contiguous"
    M, K = a.shape
```

```
    K, N = b.shape
    # Allocates output.
    c = torch.empty((M, N), device=a.device, dtype=a.dtype)

    # 1D launch kernel where each block gets its own program.
    def grid(META):
        return (triton.cdiv(M, META["BLOCK_SIZE_M"]) * triton.cdiv(N,
META["BLOCK_SIZE_N"]), )

    matmul_kernel[grid](
        a, b, c,  #
        M, N, K,  #
        a.stride(0), a.stride(1),  #
        b.stride(0), b.stride(1),  #
        c.stride(0), c.stride(1),  #
        ACTIVATION=activation,  #
    )
    return c


argparser = argparse.ArgumentParser()
argparser.add_argument("--profile", action="store_true")
argparser.add_argument("--pcsampling", action="store_true", default=False)
argparser.add_argument("--cudagraph", action="store_true", default=False)
args = argparser.parse_args()


@triton.testing.perf_report(
    triton.testing.Benchmark(
        x_names=["M", "N", "K"],  # Argument names to use as an x-axis for the
plot
        x_vals=[128 * i for i in range(2, 10)],  # Different possible values for
`x_name`
        line_arg="provider",  # Argument name whose value corresponds to a
different line in the plot
        # Possible values for `line_arg`
        line_vals=["cublas", "triton"],
        # Label name for the lines
        line_names=["cuBLAS", "Triton"],
        # Line styles
        styles=[("green", "-"), ("blue", "-")],
        ylabel="TFLOPS",  # Label name for the y-axis
        plot_name="matmul-performance",  # Name for the plot, used also as a file
name for saving the plot.
        args={},
    ))
def benchmark(M, N, K, provider):
    a = torch.randn((M, K), device="cuda", dtype=torch.float16)
    b = torch.randn((K, N), device="cuda", dtype=torch.float16)
    quantiles = [0.5, 0.2, 0.8]
    with proton.scope(f"matmul_{M}_{N}_{K}"):
        if provider == "cublas":

            @proton.scope(
                "cublas",
                metrics={
```

```
                    "flops": 2 * M * N * K,
                    "bytes": (M * N + N * K + K * M) * a.element_size(),
                },
            )
            def cublas_matmul(a, b):
                torch.matmul(a, b)

            if args.cudagraph:
                ms = triton.testing.do_bench_cudagraph(lambda: cublas_matmul(a,
b))
                min_ms = max_ms = ms
            else:
                ms, min_ms, max_ms = triton.testing.do_bench(lambda:
cublas_matmul(a, b), quantiles=quantiles)
        if provider == "triton":

            def enter_autotune(args, reset_only=False):
                if reset_only:
                    return
                proton.enter_scope("<autotune>")

            def exit_autotune(args, exception):
                proton.exit_scope()

            matmul_kernel.pre_hook = enter_autotune
            matmul_kernel.post_hook = exit_autotune
            with proton.scope("triton"):
                if args.cudagraph:
                    ms = triton.testing.do_bench_cudagraph(lambda: matmul(a, b))
                    min_ms = max_ms = ms
                else:
                    ms, min_ms, max_ms = triton.testing.do_bench(lambda:
matmul(a, b), quantiles=quantiles)

    def perf(ms):
        return 2 * M * N * K * 1e-12 / (ms * 1e-3)

    return perf(ms), perf(max_ms), perf(min_ms)


if args.profile:
    if args.pcsampling:
        # proton-viewer -m num_samples/%,time/s ./matmul.hatchet
        proton.start("matmul", hook="triton", backend="cupti_pcsampling")
    else:
        # proton-viewer -m tflop/s,time/s ./matmul.hatchet
        proton.start("matmul", hook="triton")
    benchmark.run(show_plots=True, print_data=True)
    proton.finalize()
else:
    benchmark.run(show_plots=True, print_data=True)
```

运行

```
python proton_test.py --profile
```

可视化查看

```
pip install llnl-hatchet
proton-viewer -m time/s matmul.hatchet
```